Combinatorial Algorithms (Math 482), 2026 Spring — Quiz 2

1:50-2:15pm, Monday, February 23, 2026.

1. **Substitution Method.** Prove that the recurrence $T(n) = T(\frac{3n}{4}) + 5\,T(\frac{n}{4}) + O(n^2)$ solves to $T(n) = O(n^2)$.

   Answer. Suppose that $T(n) \le T(\frac{3n}{4}) + 5\,T(\frac{n}{4}) + cn^2$ for a given constant $c > 0$. We want to show that $T(n) \le dn^2$ for some constant $d > 0$ that we can choose.

   For induction, assume that $T(k) \le dk^2$ for all $k < n$, and we need to prove $T(n) \le dn^2$.

$$\begin{aligned} T(n) &\le T\left(\frac{3n}{4}\right) + 5\,T\left(\frac{n}{4}\right) + cn^2 \\ &\le d\left(\frac{3n}{4}\right)^2 + 5\,d\left(\frac{n}{4}\right)^2 + cn^2 \\ &= \left(\frac{9d}{16} + 5\,\frac{d}{16} + c\right) n^2 \\ &= \left(\frac{7d}{8} + c\right) n^2 \le dn^2, \end{aligned}$$

   if $\frac{7d}{8} + c \le d \;\Leftrightarrow\; c \le \frac{d}{8} \;\Leftrightarrow\; 8c \le d$, that is, if we choose $d = 8c$.

2. **Recurrence Tree Method.** Consider the recurrence $T(n) = T(\frac{4n}{7}) + T(\frac{3n}{7}) + O(n)$.

   (i) What is the height of the recursion tree?
   (ii) Give an upper bound for the work done on each level.
   (iii) Calculate the summation over all levels.

   Answer. (i) The problem size is n at the root, and the largest problem on level i has size $n \cdot (4/7)^i$. We solve $n \cdot (4/7)^h = \Theta(1)$ for the height h: It gives $(7/4)^h = \Theta(n)$, or $h = \Theta(\log_{7/4} n) = \Theta(\log n)$.

   (ii) The work done on level 0 is cn. On level 1, it is $c(\frac{4n}{7}) + c(\frac{3n}{7}) = cn$. Even though the recursion tree is asymmetric, we see that the work is at most cn on every level.

   (iii) Summation over all levels gives $T(n) \le \sum_{i=0}^{h}(cn) = (h+1)(cn) = O(n \log n)$.

(TURN THE PAGE)

3. **Longest Palindrome Subsequence.** Oliver suggests the algorithm below for the Longest Palindrome Subsequence problem on a string $A[1..n]$. He tested the algorithm and it gave correct answers on the test strings such as *noon*, *level*, *cardiac*, and *services*. Prove or disprove that this algorithm always finds the optimum solution.

```
Palindrome(A[1..n])
begin
    P ⟵ 0
    i ⟵ 1
    j ⟵ n
    while i < j do
        if A[i] = A[j] then
            P ⟵ P + 2
            i ⟵ i + 1
            j ⟵ j − 1
        else
            i ⟵ i + 1
    if i=j then
        P ⟵ P + 1
    return P
```

Answer. Oliver's algorithm increments i when $A[i] \neq A[j]$, so it misses OPT if the longest palindrome subsequence includes $A[i]$ (e.g., $A[i] = A[k]$ for some $i < k < j$). For example, if $A[1..3] =$*eel*, it returns $P = 1$, but the longest palindrome is *ee*.

4. **Subset Sum without Consecutive Pairs.** Consider the following modified version of the subset sum problem. Given an array $A[1..n]$ of positive integers and a capacity W, we need to choose a subset to maximize the sum subject to the capacity constraint W such that no two consecutive items are chosen (we cannot choose both $A[i]$ and $A[i+1]$).

   Let $DP[i, j]$ denote the maximum sum using $A[1..i]$ subject to capacity constraint j, and such that no two consecutive items are chosen. Write a recurrence formula for $DP[i, j]$.

Answer. In general, suppose that $A[i] \leq j$, so we have two options: We either include $A[i]$ in the sum or not. If we exclude $A[i]$, we can recurse on $A[1..(i-1)]$ and the capacity does not change. If we include $A[i]$, then we have to skip $A[i-1]$, so we recurse on $A[1..(i-2)]$ and the capacity decreases by $A[i]$. Take the best of the two options:

$$D[i, j] = \max\big(D[i-1, j], A[i] + D[i-2, j - A[i]]\big).$$